

Harnessing Machine Learning for Breast Cancer Prediction and Diagnosis

Higher National Diploma in Software Engineering 24.2F

Machine Learning

Coursework Report

SUBMITTED BY

Group 05

COHNDSE242F-005	GANGADARA M K D
COHNDSE242F-008	JAYARATNE A H V M
COHNDSE242F-019	JAYASINGHE D S
COHNDSE242F-040	WEERASOORIYA W A U S

Date of Submission: September 19, 2025



School of Computing and Engineering
National Institute of Business Management
Colombo-7

Table of Contents

1. Introduction.....	3
2. Problem Definition.....	4
3. Data Collection	5
4. Data Preprocessing.....	6
5. Model Selection	18
6. Model training.....	20
6.1 Logistic regression	20
6.2 Random Forest	21
6.3 Random Forest with Cross validation.....	25
6.4 Model with gradient boosting	32
7. Model Evaluations	37
7.1 Logistic regression	37
7.2 Random Forest	40
7.3 Random Forest with Cross validation.....	42
7.4 Model with gradient boosting	44
8. Deployment.....	46
9. Conclusion	48
References.....	49

Git Hub link: https://github.com/mkdinindu/ML_Course_Work.git

1. Introduction

Breast cancer is one of the most common and deadliest cancers in women all over the world, and it penetrates into families, societies, and health systems (World Health Organization, 2025). It is caused by abnormal breast cells beginning to grow out of control, often as tumours within the lobules (glands that produce milk) or the ducts (the tubes which have milk) (World Health Organization, 2025). Even medical progress has brought better treatment outcomes, early detection of cancer remains the most vital factor for improving survival rates (G.M, Alkabban & Ferguson, 2024).

The current diagnostic process requires patients to undergo invasive tests including biopsies and fine-needle aspirations before pathologists conduct their analysis (Wolberg, Street, Heisey, & Mangasarian, 1995). The current diagnostic methods prove reliable, yet they require extensive time and resource costs and sometimes produce results that depend on human judgements. The increasing use of medical imaging and computer-based diagnostic tools requires healthcare professionals to develop automated methods which provide precise and scalable breast tumor classification support.

This report illustrates a project of using machine learning techniques in the significant task of classifying breast tumours. From digitized images of fine-needle aspirates, by analyzing characteristics of cell nuclei, our goal is to create a model that can accurately label benign (non-malignant) and malignant (cancerous) tumours. The project not only delves into the path from raw data preprocessing to model training and evaluation but also proves that powerful algorithms such as Random Forest Classifiers can yield strong, interpretable, and highly accurate predictions.

The utility of this work lies in its potential to enhance clinical decision-making, reduce diagnostic delay, and lower errors that ultimately result in quicker interventions for patients. This project also demonstrates an end-to-end machine learning model building experience from choosing datasets and exploratory data analysis to model tuning and deployment making it an effective academic exercise and demonstration of real-world use.

In essence, the report bridges the gap between medicine and technology by demonstrating how strategies based on data can progress medical professionals in the fight against breast cancer.

2. Problem Definition

Breast cancers start due to faulty cells that grows uncontrollably, stop obeying gene coded rules. Most of times these tumours starts to grow within Lobules (glands that produce milk) or within Ducts (called as Ductal carcinoma).

Nevertheless, early detection significantly improves survival rates of patients.

Most of times traditional diagnostic methods, rely on invasive procedures, and expert medical judgment which will be time consuming and sometimes prone to human error.

This project aims to develop and fine tune a machine learning model to automatically classify breast tumours as **benign** (non-cancerous) or **malignant** (cancerous) based on sample cell nuclei features obtained from digitized images of fine-needle aspirate procedures.

Project objectives are

- Finding the best suited feasible dataset.
- Data preprocessing
- Exploratory data analysis – understand data distribution and correlations
- Model development
- Testing and validation
- Fine tuning the model

3. Data Collection

To accomplish our project objectives and train a model, we chose a freely available open-source dataset from Kaggle.com.

<https://www.kaggle.com/datasets/imtkaggleteam/breast-cancer>

This dataset is consists of data points gathered from 569 instances. These data features are computed from digitized images of fine needle aspirated breast mass samples. These features describe the characteristics of the cell nuclei present in that image.

Data describing

- This dataset consists of 32 columns.
- id
- diagnosis
- Rest of them are 10 distinct features in 3 forms.
 1. Mean value
 2. Standard error
 3. Worst (largest value observed)

Main features

[radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension]

Mean features

[radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean, compactness_mean, concavity_mean, concave points_mean, symmetry_mean, fractal_dimension_mean]

Worst features

[radius_worst, texture_worst, perimeter_worst, area_worst, smoothness_worst, compactness_worst, concavity_worst, concave points_worst, symmetry_worst, fractal_dimension_worst]

Standard error features

[radius_se, texture_se, perimeter_se, area_se, smoothness_se, compactness_se, concavity_se, concave points_se, symmetry_se, fractal_dimension_se]

4. Data Preprocessing

Data preprocessing is the process of cleaning and removing unwanted features in the data set to make the dataset much more usable and effective for more accurate predictions. The model's quality is heavily depends on the quality of dataset which the model is trained on.

There might be different type of data processing needs for different type of scenarios. If there are missing values in the data set, filling those values should be done by a suitable method like imputation or the number of missing values is larger the column can be removed. Categorical variables can be handled by Ordinal Encoding or One-Hot Encoding.

Another considerable point is there can be noisy data which leads the model in wrong directions when making predictions.

Data preprocessing is mostly valuable for prevent target data leakage.

“**Target leakage** occurs when your predictors include data that will not be available at the time you make predictions. It is important to think about target leakage in terms of the *timing or chronological order* that data becomes available, not merely whether a feature helps make good predictions.” (cook, 2025)

Hence data preprocessing is one of the major tasks when training a machine learning model with ladled data.

- Importing Python library Pandas and the Cancer dataset.
- Displaying the first 5 rows.

```
import pandas as pd

# Read the data
cancerData = pd.read_csv("Cancer_Data.csv")
cancerData.head()

✓ 12.8s
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...

5 rows × 33 columns

- Importing necessary libraries for plotting data.
- Displaying the dataset as a multi-line plot visualization.
 - X-axis – Display each data point
 - Y-axis – Shows the numerical value of each feature for those data points

```

pd.plotting.register_matplotlib_converters()
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns

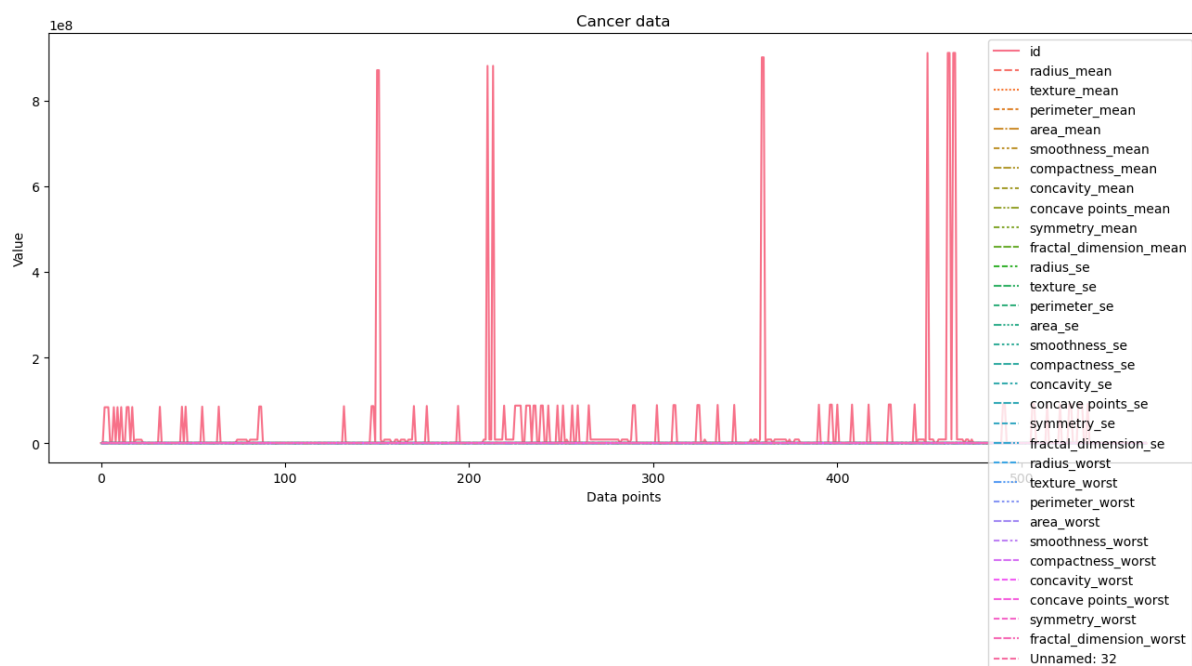
# Set the width and height of the figure
plt.figure(figsize=(16,6))

# Add title
plt.title("Cancer data")

# Line chart showing how values and datapoints
sns.lineplot(data=cancerData)

# Add label for horizontal axis
plt.ylabel("Value")
plt.xlabel("Data points")
plt.show()
✓ 21.5s

```



In this plot it is visible that 31 features which are grouped into three main categories and one unnamed column.

Among these features “id” is a unique identifier, hence not useful for training. Unnamed: 32 is an empty column. Hence those columns are decided to be dropped.

In order to decide if there are any other columns to drop that doesn’t affect the outcome in a meaningful way we did conduct a t-test and data visualization. It helped us to see how each feature differs between malignant and benign tumours.

Pandas, matplotlib, seaborn, and scipy are the libraries used in this test.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ttest_ind

# Load the data
df = pd.read_csv("Cancer_Data.csv")

# Preprocess the target variable
df['diagnosis_numeric'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Define the features to check
features = df.drop(['diagnosis', 'Unnamed: 32', 'diagnosis_numeric'], axis=1).columns

# Set up the plotting environment
plt.style.use('seaborn-v0_8-whitegrid')

# Loop through each feature to create visualizations and perform t-tests
for feature in features:
    # Create box plots
    plt.figure(figsize=(8, 6))
    sns.boxplot(x='diagnosis', y=feature, data=df)
    plt.title(f'Distribution of {feature} by Diagnosis', fontsize=14)
    plt.xlabel('Diagnosis', fontsize=12)
    plt.ylabel(feature, fontsize=12)
    plt.tight_layout()
    plt.savefig(f'picDir/{feature}_boxplot.png')
    plt.close()

    # Perform t-test
    group_M = df[df['diagnosis'] == 'M'][feature]
    group_B = df[df['diagnosis'] == 'B'][feature]

    t_stat, p_value = ttest_ind(group_M, group_B, equal_var=False)

    print(f"Feature: {feature}")
    print(f"  T-statistic: {t_stat:.4f}")
    print(f"  P-value: {p_value:.10f}")
    if p_value < 0.05:
        print("  Conclusion: The difference in means is statistically significant (p < 0.05).")
    else:
        print("  Conclusion: The difference in means is not statistically significant (p >= 0.05).")
    print("-" * 50)
```

This test produced 30 separate outputs for each feature. Below are some of the t-test results returned from this test.

```

Feature: id
T-statistic: 0.9086
P-value: 0.3641345284
Conclusion: The difference in means is not statistically significant (p >= 0.01).
-----
Feature: radius_mean
T-statistic: 22.2088
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: texture_mean
T-statistic: 11.0221
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: perimeter_mean
T-statistic: 22.9353
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: area_mean
T-statistic: 19.6410
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: smoothness_mean
T-statistic: 9.2974
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: compactness_mean
T-statistic: 15.8182
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: concavity_mean
T-statistic: 20.3324
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: concave points_mean
T-statistic: 24.8448
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: symmetry_mean
T-statistic: 8.1122
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).
-----
Feature: fractal_dimension_mean
T-statistic: -0.2969
P-value: 0.7667216011
Conclusion: The difference in means is not statistically significant (p >= 0.01).
-----
Feature: radius_se
T-statistic: 13.3007
P-value: 0.0000000000

```

```

Feature: radius_se
T-statistic: 13.3007
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).

Feature: texture_se
T-statistic: -0.2079
P-value: 0.8354170682
Conclusion: The difference in means is not statistically significant (p >= 0.01).

Feature: perimeter_se
T-statistic: 12.8328
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).

Feature: area_se
T-statistic: 12.1556
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).

Feature: smoothness_se
T-statistic: -1.6229
P-value: 0.1052970030
Conclusion: The difference in means is not statistically significant (p >= 0.01).

Feature: compactness_se
T-statistic: 7.0826
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).

Feature: concavity_se
T-statistic: 6.9163
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).

Feature: concave points_se
T-statistic: 10.7355
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).

Feature: symmetry_se
T-statistic: -0.1421
P-value: 0.8871222603
Conclusion: The difference in means is not statistically significant (p >= 0.01).

Feature: fractal_dimension_se
T-statistic: 2.0362
P-value: 0.0422023771
Conclusion: The difference in means is not statistically significant (p >= 0.01).

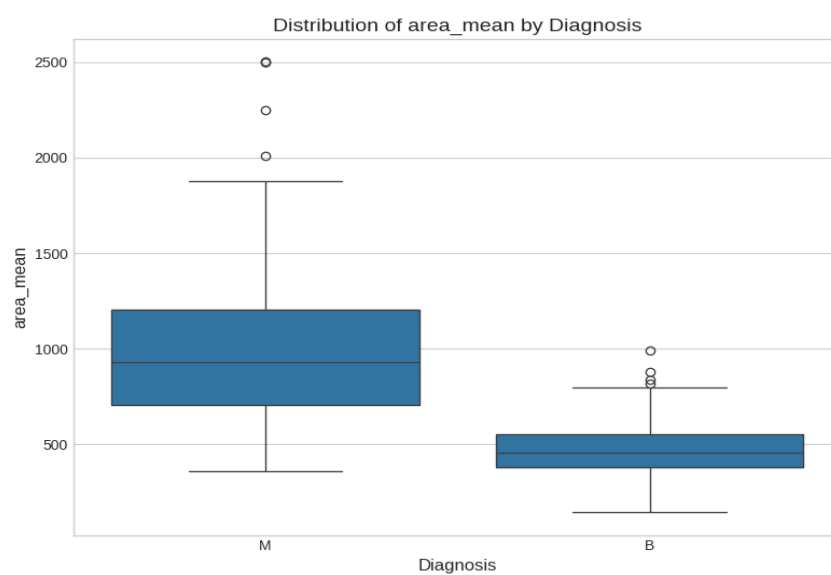
Feature: radius_worst
T-statistic: 24.8297
P-value: 0.0000000000
Conclusion: The difference in means is statistically significant (p < 0.01).

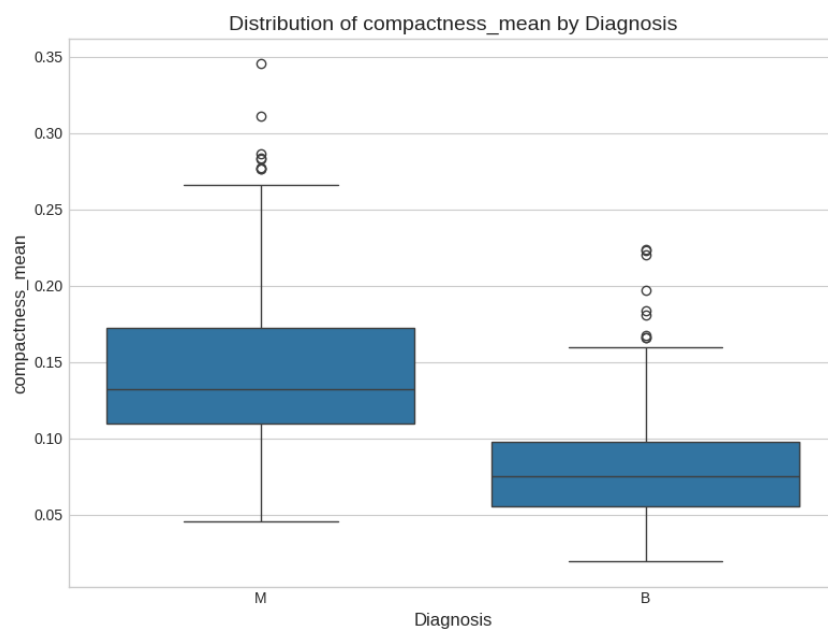
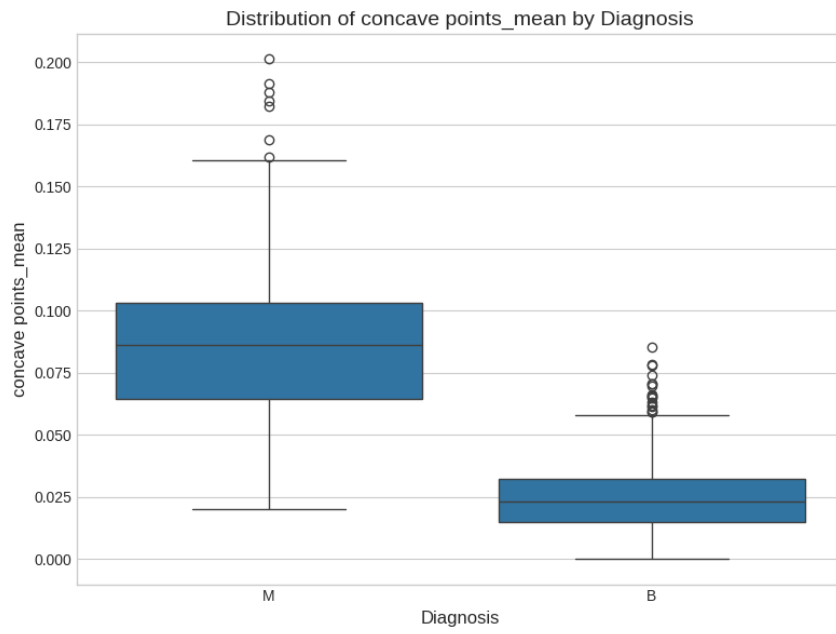
Feature: texture_worst
T-statistic: 12.2648

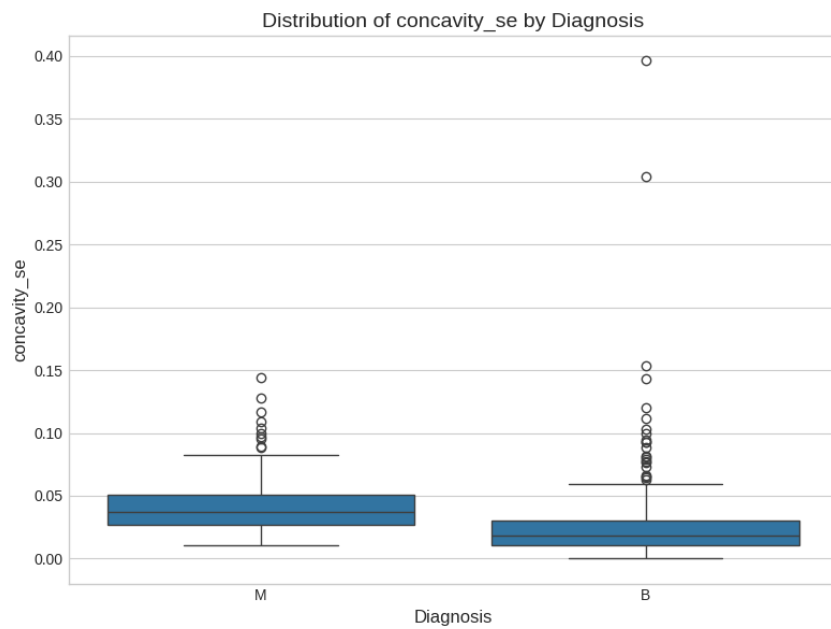
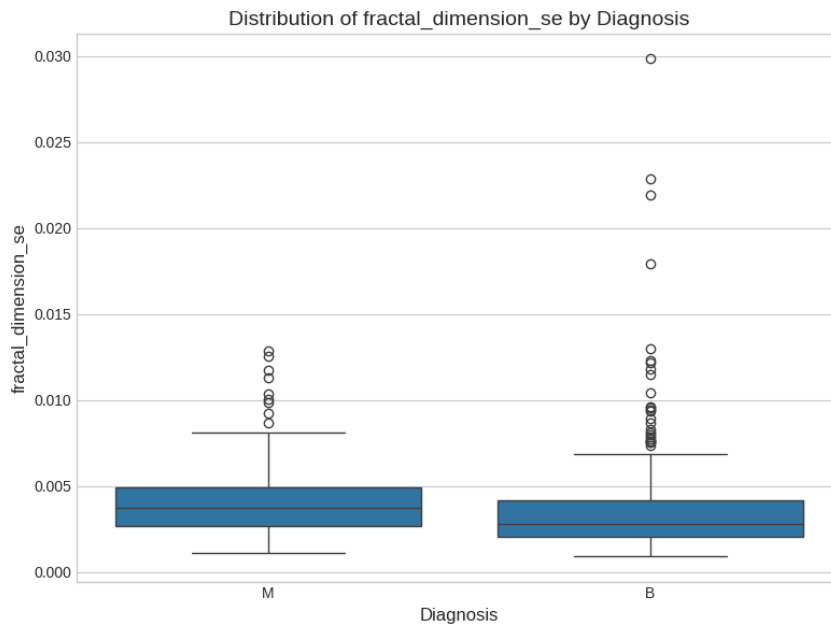
```

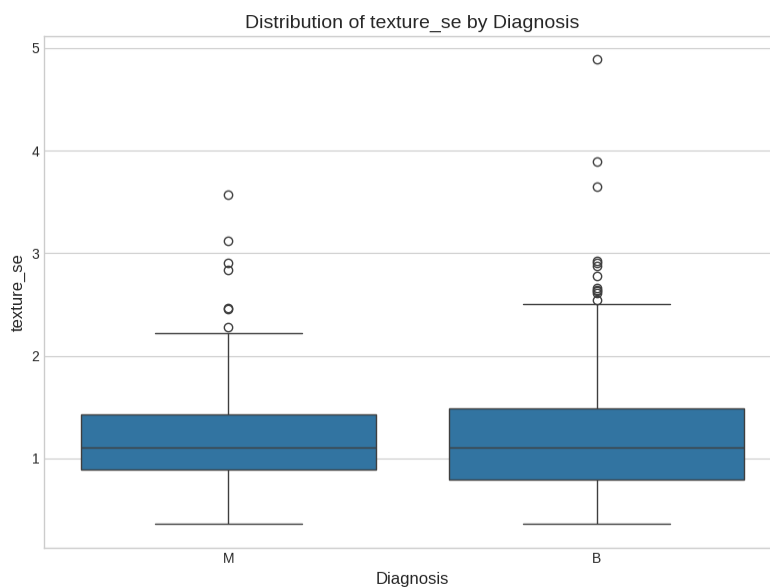
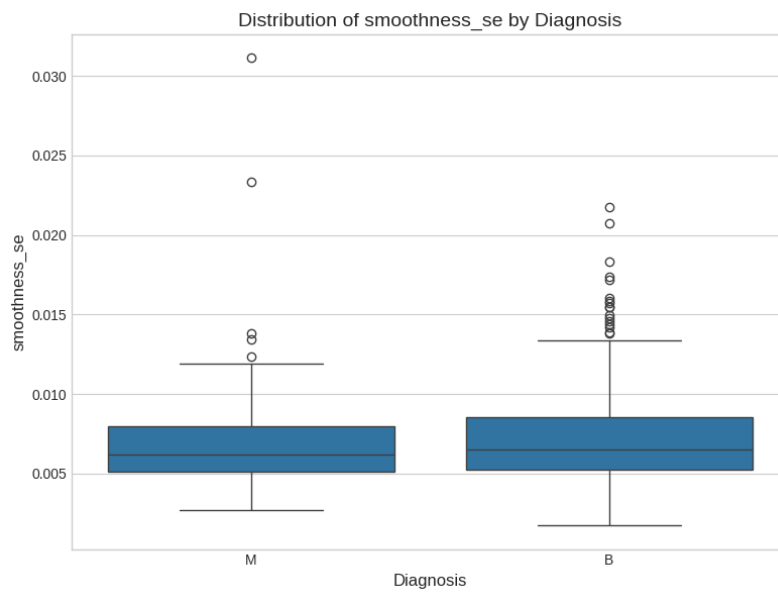
The visualized images generated by this code will be saved in a directory called a 'picDir'. This decision was made to prevent Google colab environment file system from getting cluttered.

Below are some of visualized images









As you can visually observe few features in this dataset doesn't strongly correlate positively or negatively with output. We considered p values given by our t-tests and decided features with more than 0.01 p value are not important, hence, we decided to drop that columns to prevent noise in the dataset.

- Mapping the output (“diagnosis”) values ‘M’ as 1 and ‘B’ as 0.

```
cancerData['diagnosis'] = cancerData['diagnosis'].map({"M":1, "B":0})
```

✓ 0.0s

- Separating labelled data into output (y) and features (X).
- Dropping unwanted features.

```
# Separate target from predictors
y = cancerData.diagnosis
X = cancerData.copy()
# X.drop(['diagnosis', 'id', 'Unnamed: 32', 'texture_se', 'smoothness_se'], axis=1, inplace=True)
X.drop(['diagnosis', 'id', 'concavity_se', 'fractal_dimension_se', 'smoothness_se', 'texture_se', 'Unnamed: 32'], axis=1, inplace=True)
```

✓ 0.0s

- Displaying first five rows of the features (X) after data preprocessing.

```
X.head()
```

✓ 0.0s

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809

5 rows × 26 columns

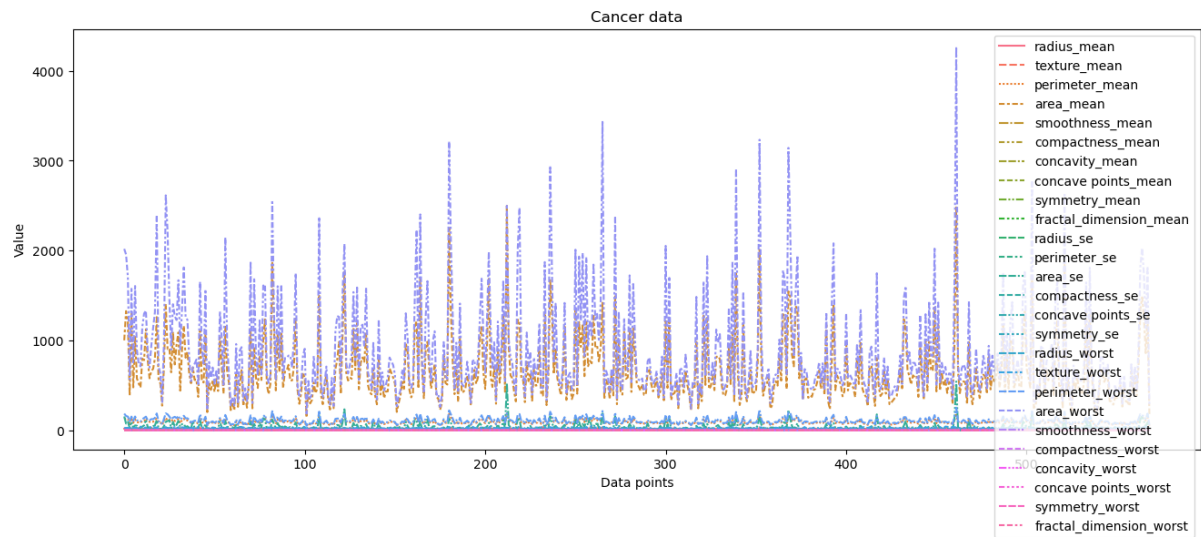
- Displaying features (X) as a multi-line plot visualization after data preprocessing.

```
# Set the width and height of the figure
plt.figure(figsize=(16,6))

# Add title
plt.title("Cancer data")

# Line chart showing how values and datapoints
sns.lineplot(data=X)

# Add label for horizontal axis
plt.ylabel("Value")
plt.xlabel("Data points")
plt.show()
```

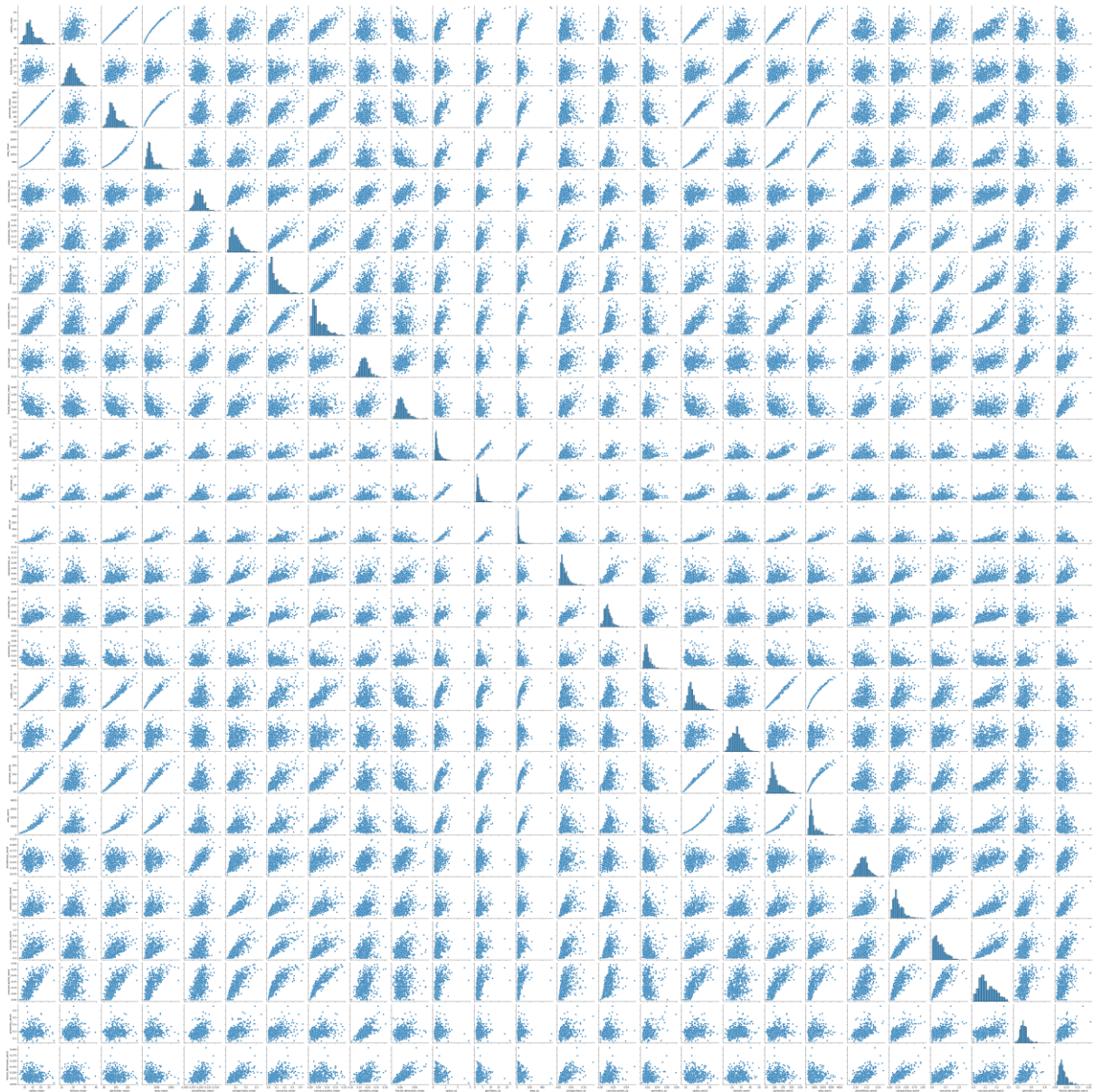


This plot is cleaner and more focused on actual diagnostic features considering to the previous visualisation.

- Visualising features in a pair plot.

```
sns.pairplot(X)
✓ 2m 19.0s
<seaborn.axisgrid.PairGrid at 0x240d59c46e0>

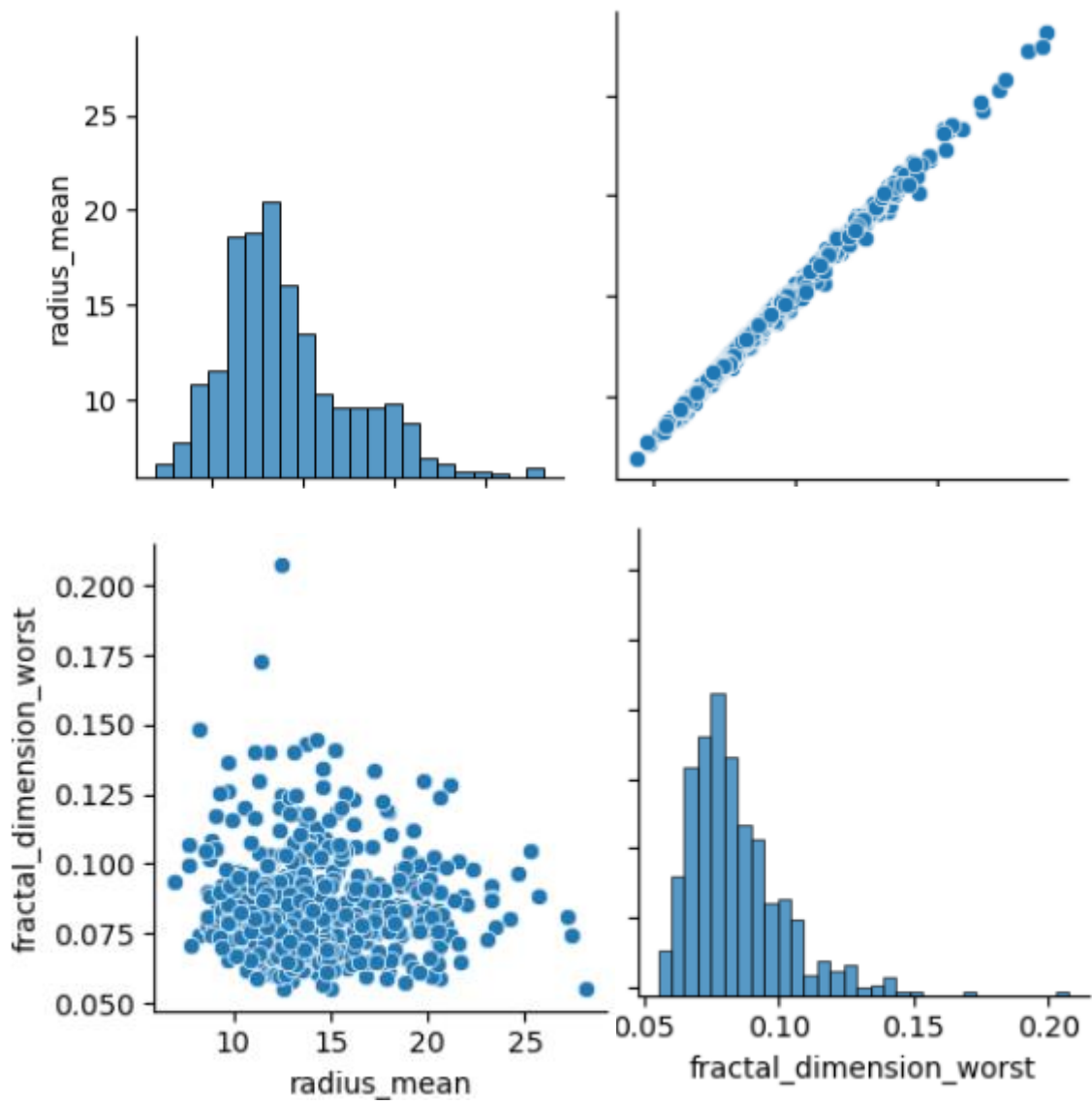
plt.show()
✓ 43.6s
```



In this pair plot Grid of plots comparing every variable in the dataset against every other variable.

Diagonal cells are the histograms which shows the distribution of each individual feature.

In the other hand Off diagonal cells are scatter plots showing how two features relate to each other.



5. Model Selection

When training a machine learning model for making predictions for a specific task, selecting a suitable model is crucial. There are different types of machine learning models based on the type of output that wanted to predict.

In supervised learning, for regression tasks where the goal is to predict any numerical (Continues) values Linear regression, Ridge Regression, Decision Tree Regressor and Random Forest Regressor can be used while Logistic Regression, Decision Tree Classifier and Random Forest Classifier can be used when model is used for predicting any categorical values.

We decided to train and tune models using, different algorithms and choose the best one among them.

Chosen model algorithms are,

- Logistic regression
- Random forest
- Random forest with cross validation
- XGB boost

Why choose logistics regression

- **Simplicity & Interpretability** – Provides clear coefficients that show how each feature influences the probability of an outcome.
- **Probabilistic Output** – Predicts probabilities (not just classes), useful in risk-based decisions (e.g., medical diagnosis).
- **Efficiency** – Fast to train and works well with linearly separable data or small-to-medium datasets.
- **Baseline Model** – Often used as a starting point to benchmark performance before trying more complex models.

Why Choose Random Forest

- High Accuracy
 - Random Forest calculate predictions from multiple decision trees, get the highest vote as the decision reducing variance.
- Handle Complex feature Interactions
 - Can model non-linear relationships and feature interactions with large amounts of features without explicit transformation.
- Resistant to overfitting
 - Bootstrap aggregation prevents the model from leaning on noisy data.
- Feature importance insights
 - Ranks features by importance.
- Works well with imbalanced data
- Scalable and Fast

Why choose random forest with cross validation

- **Handles Complexity & Non-linearity** – Captures non-linear relationships and interactions between features.
- **Robustness** – Reduces overfitting compared to single decision trees by averaging multiple trees.
- **Feature Importance** – Provides insight into which features matter most for prediction
- **Reliable Performance via Cross-Validation** – Cross-validation ensures model performance is not due to chance and generalizes well on unseen data.

Why choose XG Boost

- **High Accuracy** – Often outperforms other algorithms in structured/tabular datasets (common in Kaggle competitions).
- **Speed & Efficiency** – Optimized implementation of gradient boosting; supports parallel processing and handles large datasets well.
- **Flexibility** – Works for regression, classification, ranking, and supports missing values directly.

6. Model training

6.1 Logistic regression

- Importing pandas library and refering it as 'pd'.
- Reading the dataset and copy it into pandas data frame named 'df'.
- Displaying some of top values from df.

```
import pandas as pd

df = pd.read_csv("/content/Cancer_Data.csv")
df.head()
```

- Mapping M as 1 and B as 0

```
df['diagnosis'] = df['diagnosis'].map({"M":1, "B":0})
```

- Dropping columns decided as unwanted.
- Setup X with selected features.
- Setup y with targeted column.

```
X = df.drop(['id', 'Unnamed: 32', 'diagnosis', 'concavity_se', 'fractal_dimension_se', 'smoothness_se', 'texture_se'], axis=1)

y = df['diagnosis']
```

- Importing Standard scaler library from sklearn.
 - Standard scaling is done because there are some features like area mean has huge values like '1001' while other features has low values like '0.11'. This indifference could cause model to prioritize large values, which we don't want to be happened.

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
print(X_scaled)

[[ 1.09706398 -2.07333501  1.26993369 ...  2.29607613  2.75062224
  1.93701461]
 [ 1.82982061 -0.35363241  1.68595471 ...  1.0870843  -0.24388967
  0.28118999]
 [ 1.57988811  0.45618695  1.56650313 ...  1.95500035  1.152255
  0.20139121]
 ...
 [ 0.70228425  2.0455738   0.67267578 ...  0.41406869 -1.10454895
 -0.31840916]
 [ 1.83834103  2.33645719  1.98252415 ...  2.28998549  1.91908301
  2.21963528]
 [-1.80840125  1.22179204 -1.81438851 ... -1.74506282 -0.04813821
 -0.75120669]]
```

- Importing train test splitting library from sklearn
- Importing a Logistic regression model from sklearn.
- Importing accuracy-score, confusion matrix and classification report libraries from sklearn

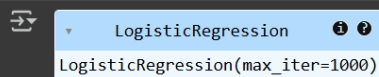
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

- Splitting dataset in to train and test.
- 20% of dataset will be used to train the model.
- Random state used is 42. Random state is what that helps to recreate the exact session.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

- Declaring a Logistic regression model called 'model'.
- Setting the max iteration level for the model.
- Fit the model with training features and target variable.

```
model = LogisticRegression(max_iter=1000) # max_iter increased to ensure convergence
model.fit(X_train, y_train)
```



- Predicting test dataset using the model and assigning output to the y_pred.

```
y_pred = model.predict(X_test)
```

6.2 Random Forest

- Importing train_test_split from sklearn library.
- Splitting data set into training and validating(testing) data.

```
# Break off validation set from training data

from sklearn.model_selection import train_test_split

X_train, X_valid, y_train, y_valid = train_test_split(X, y, train_size= 0.8, test_size= 0.2, random_state=0)
```

X_train.head()

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean
338	10.05	17.53	64.41	310.8	0.10070	0.07326	0.02511	0.01775	0.1890
427	10.80	21.98	68.79	359.9	0.08801	0.05743	0.03614	0.01404	0.2016
406	16.14	14.86	104.30	800.0	0.09495	0.08501	0.05500	0.04528	0.1735
96	12.18	17.84	77.79	451.1	0.10450	0.07057	0.02490	0.02941	0.1900
490	12.25	22.44	78.18	466.5	0.08192	0.05200	0.01714	0.01261	0.1544

5 rows × 10 columns

- Importing necessary libraries for specific tasks like creating pipeline, numerical data processing, calculating MAE and Random Forest Classifier.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import mean_absolute_error
```

- Define a processor for processing numerical data with the simple imputer if any further processing is needed. (Filling any missing values with its mean if there is any.)

```
# Preprocessing for numerical data
numerical_transformer = SimpleImputer(strategy='constant')
```

- Define a model with Random Forest Classifier

```
# Define model
model = RandomForestClassifier(n_estimators=100, random_state=0)
```

- Bundling a pipeline with processor and model we defined.

```
# Bundle preprocessing and modeling code in a pipeline
clf = Pipeline(steps=[('preprocessor', numerical_transformer),
                       ('model', model)])
```

- Executing pipeline to process and train the model step by step.

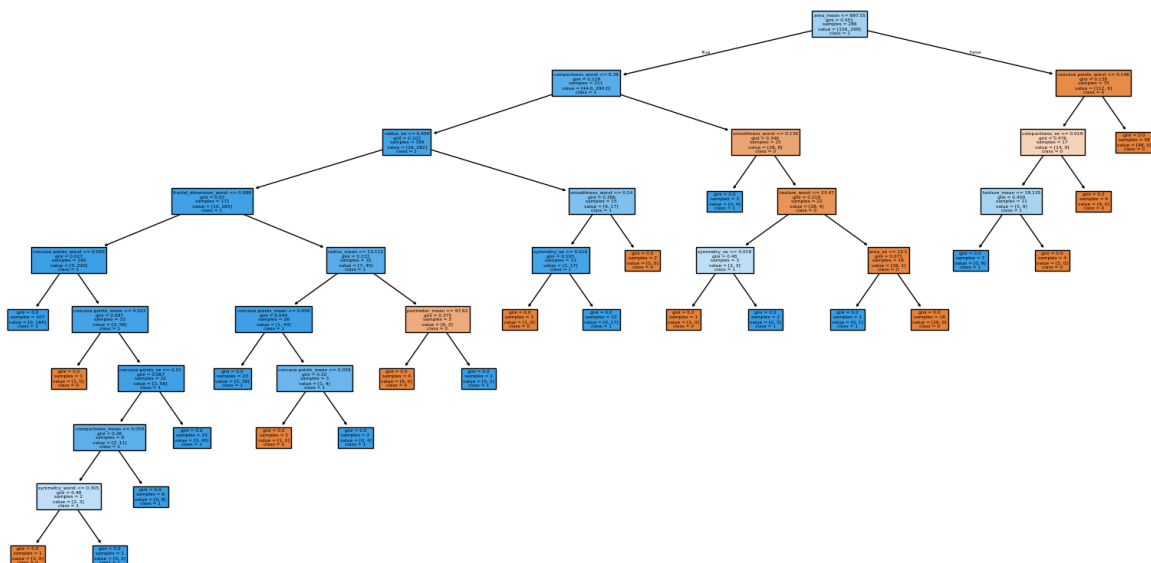


- Visualize the first decision tree from the Random Forest.

```
# visualize a single decision tree from the forest

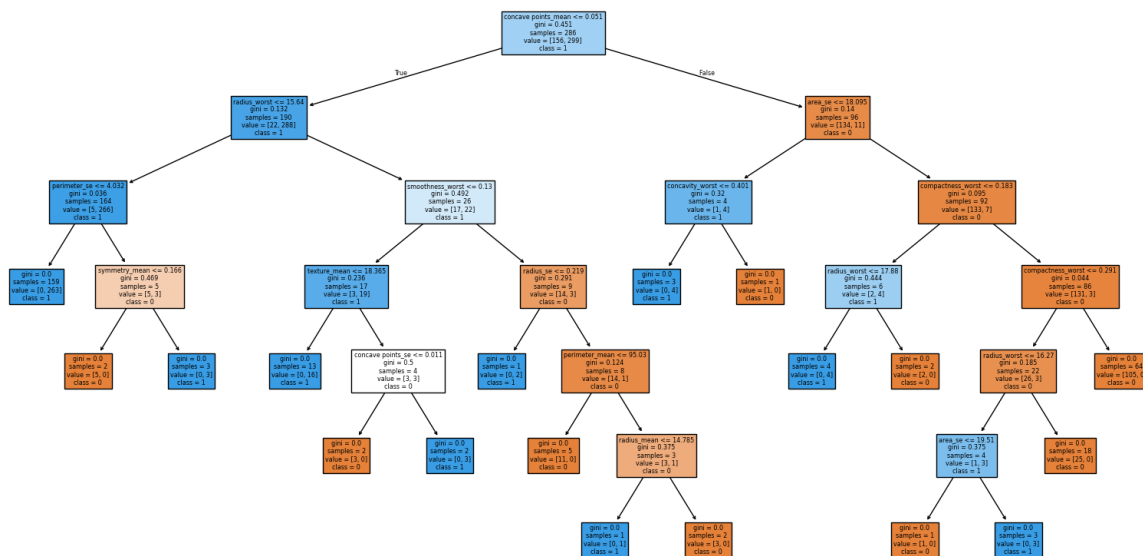
from sklearn.tree import plot_tree

# Visualize the first tree
plt.figure(figsize=(20, 10))
plot_tree(model.estimators_[0], #change value to see a different tree
          filled=True,
          feature_names=X.columns,
          class_names=[str(cls) for cls in model.classes_]) # Convert to strings
plt.show()
```



- Visualize the second decision tree from the Random Forest.

```
# Visualize the second tree
plt.figure(figsize=(20, 10))
plot_tree(model.estimators_[1], #change value to see a different tree
          filled=True,
          feature_names=X.columns,
          class_names=[str(cls) for cls in model.classes_]) # Convert to strings
plt.show()
```



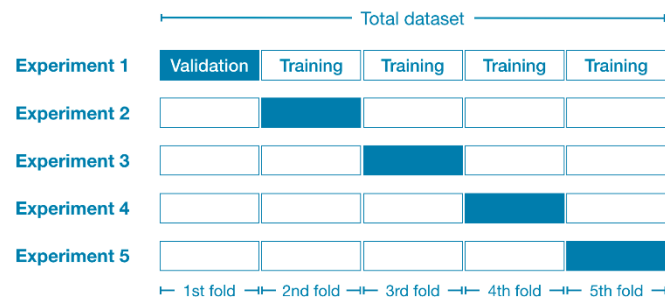
- Making predictions with the trained Random Forest model.

```
# Get predictions from validating data
preds = clf.predict(X_valid)
```

6.3 Random Forest with Cross validation

Cross validation is a method of running the modelling process by dividing the data set into different subsets to get multiple measures of the model quality.

The number of subsets which the data set is divide into, can be decided by the developers as their will. The subsets are also known as “folds”. If the number of folds is 5, one subset contains 20 percent of the data from the whole data set.



Then the experiment for each fold can be run.

- In first experiment, first fold is used as validation (or holdout) data and the rest is used for the training process. This gives a measure of model quality based on a 20 percent holdout set.
- In Second experiment, second fold is used as validation data while the rest can be used for training the model. The holdout set is then used to get a second estimate of model quality.
- Repeating this process, using every fold once as the holdout set. After combining all results will give a measure of model quality that is based on all the rows in data set, which 100 percent of data is used as holdout at some point.

The accuracy of the model can be differed according to the number of folds that data is divided into.

- Writing a function to obtain the best number of folds in the range of 2 to 9.
- The declared loop inside the function will iterate 8 times for execute the `cross_val_score()` function to execute the pipeline, make prediction and calculate the mean absolute error.
- In every iteration the number of folds will changed from 2 to 9.
- After every iteration the mean absolute error of every model is checked and after the execution ends the least mean absolute error will print with the number of folds.

```
# Uses the cross_val_score() function to obtain the mean absolute error (MAE), averaged across i (cv parameter) different folds.
# in a iteration of 8 times to get the best mae with the amount of folds

from sklearn.model_selection import cross_val_score

best_mae = float('inf')
best_cv = None

for i in range(2, 9):
    scores = -1 * cross_val_score(my_pipeline, X, y, cv=i, scoring='neg_mean_absolute_error')
    mean_score = scores.mean()
    if mean_score < best_mae:
        best_mae = mean_score
        best_cv = i

print(f"Best Average MAE: {best_mae} with cv = {best_cv}")
```

✓ 9.9s

Best Average MAE: 0.03157396653331613 with cv = 7

The least mean absolute error is printed as 0.029810 which belongs to 7 number of folds. That can be used to train a model with a good accuracy for best predictions.

- Execute the function `cross_val_score()` with the best number of folds to train the model.

```
# Uses cross_val_score() function to obtain the mean absolute error (MAE), averaged across 7 (best_cv) different folds.

from sklearn.model_selection import cross_val_score

# Multiply by -1 since sklearn calculates *negative* MAE
scores = -1 * cross_val_score(my_pipeline, X, y,
                              cv=best_cv,
                              scoring='neg_mean_absolute_error')

print("Average MAE score:", scores.mean())
```

✓ 1.9s

Average MAE score: 0.03157396653331613

- Writing a function `get_score(n_estimators)` to train the model with different `n_estimators` values. `n_estimators` refers to the number of decision trees trained inside the random forest.

```
# Function get_score() that reports the average over 7 cross-validation folds MAE
# of a machine learning pipeline

def get_score(n_estimators):
    my_pipeline = Pipeline(steps=[('preprocessor', numerical_transformer),
                                   ('model', RandomForestClassifier(n_estimators, random_state=0))
                                   ])
    scores = -1 * cross_val_score(my_pipeline, X, y,
                                   cv=best_cv,
                                   scoring='neg_mean_absolute_error')

    return scores.mean()
```

- Writing a loop for execute the `get_score(n_estimators)` function with multiple values that train 8 random forests with different number of trees.
- The mean absolute error of each forest will be stored in a python dictionary.

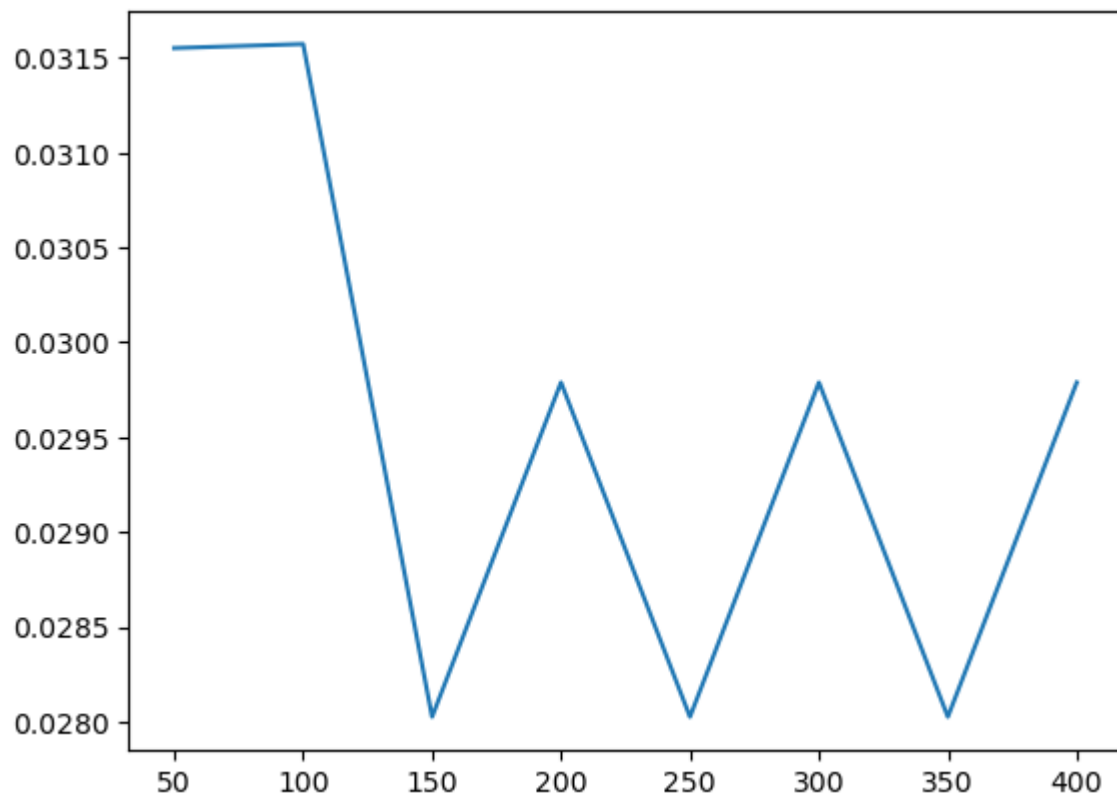
```
# Store results in a Python dictionary, where results[i] is the average MAE returned by get_score(i).

results = {}
for i in range(1,9):
    results[50*i] = get_score(50*i)
```

- Display mean absolute error of each forest in a plot.

```
import matplotlib.pyplot as plt
%matplotlib inline

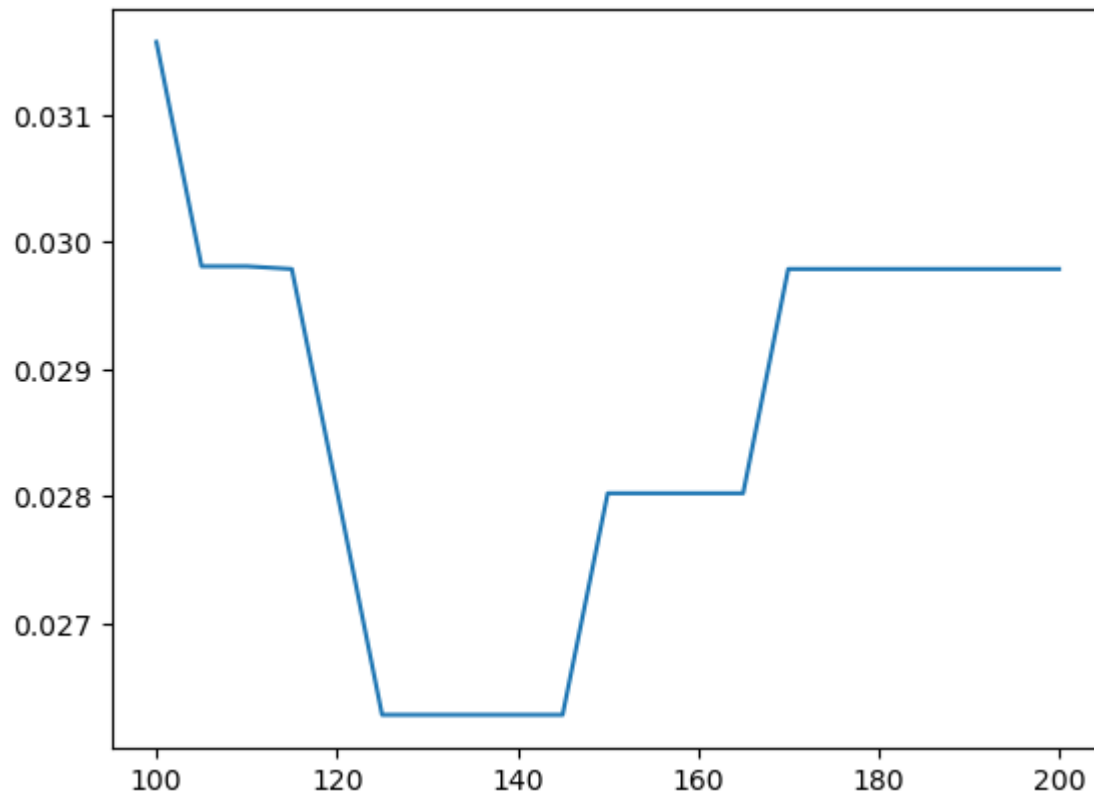
plt.plot(list(results.keys()), list(results.values()))
plt.show()
```



As the plot shows there are three minimum values for the mean absolute error. Which are 150, 250, 350. Among those three values, 150 can be chosen considering about the less time complexity and the less space complexity required for training a smaller number of trees rather than 250 or 350.

- Writing a loop for finding the best number of trees between 100 and 200 for Random Forest training.

```
# Store results in a Python dictionary, where results[i] is the average MAE returned by get_score(i).  
  
results = {}  
for i in range(100, 201, 5):  
    results[i] = get_score(i)  
✓ 1m 9.8s
```



As the least mean absolute error is between 125 and 145 number of trees. The model can be trained on `n_estimators` as 140 for a best accuracy model.

- Declaring a function for execute all this functions in one execution with calculating accuracy and plot the prediction outputs in a heatmap.

```

from sklearn.model_selection import cross_val_score, cross_val_predict
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import confusion_matrix
import seaborn as sns

def get_score_final(n_estimators):
    my_pipeline = Pipeline(steps=[('preprocessor', numerical_transformer),
                                   ('model', RandomForestClassifier(n_estimators, random_state=0))
                                   ])

    # MAE scoring (custom scorer for classification)
    mae_scores = -1 * cross_val_score(my_pipeline, X, y,
                                       cv=best_cv,
                                       scoring='neg_mean_absolute_error')

    # Accuracy scoring
    accuracy_scores = cross_val_score(my_pipeline, X, y, cv=best_cv, scoring='accuracy')

    # Cross-validated predictions for confusion matrix and classification report
    preds = cross_val_predict(my_pipeline, X, y, cv=best_cv)
    cm = confusion_matrix(y, preds)

    print(f"Cross-validated Accuracy: {accuracy_scores.mean()}")
    print(f"Cross-validated MAE: {mae_scores.mean()}")
    print("\nConfusion Matrix:\n", cm)
    print("\nClassification Report:\n", classification_report(y, preds))

    # Plot heatmap
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel('Predicted Labels')
    plt.ylabel('True Labels')
    plt.title('Confusion Matrix Heatmap (RandomForestClassifier)')
    plt.tight_layout()
    plt.show()

```

- Executing the function with best number of trees.

```
# Exectue the get_score_final() with the best n_estimator value
get_score_final(140)
```

✓ 13.3s

Cross-validated Accuracy: 0.9737170387576891
 Cross-validated MAE: 0.026282961242310835

Confusion Matrix:
 [[352 5]
 [10 202]]

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	357
1	0.98	0.95	0.96	212
accuracy			0.97	569
macro avg	0.97	0.97	0.97	569
weighted avg	0.97	0.97	0.97	569

6.4 Model with gradient boosting

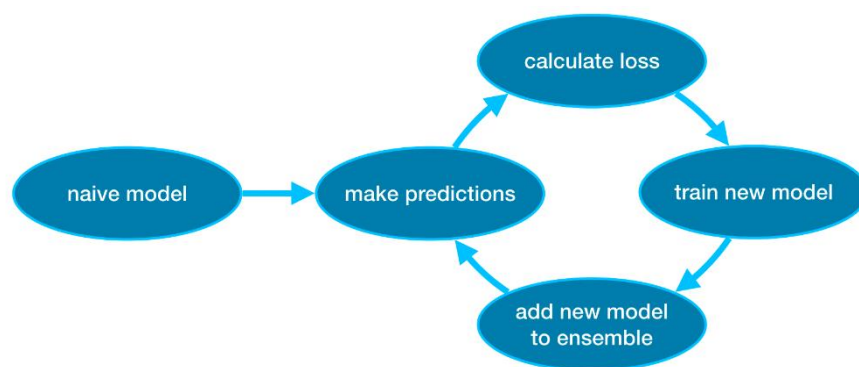
“**Gradient boosting** is a method that goes through cycles to iteratively add models into an ensemble.” (AlexisBCook, 2025)

Begin with assembling a single model with initial values for the ensemble, which will predict poor outputs with given inputs. The predictions can be used to calculate the loss function that will be a higher value in most case because of the naive model with initial values.

Then, we use the loss function to fit a new model that will be added to the ensemble. Specifically, we determine model parameters so that adding this new model to the ensemble will reduce the loss.

The "gradient" in "gradient boosting" refers to the fact that gradient decent is used on the loss function to determine the parameters in this new model.

Afterward Gradient boosting will adjust parameters until it finds the most suitable parameters for obtain the least loss function for the model.



- Importing scikit-learn API for XGBoost and define a XGBClassifier model

```
from xgboost import XGBClassifier

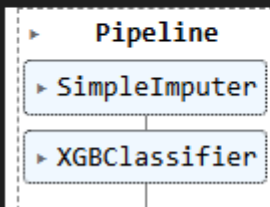
# Define model
model_1 = XGBClassifier(random_state=0)
```

- Bundling preprocessor and the model to pipeline.

```
# Bundle preprocessing and modeling code in a pipeline
Pipeline_1 = Pipeline(steps=[('preprocessor', numerical_transformer),
                              ('model', model_1)
                              ])
```

- Execute the pipeline to train the model.

```
# train the model
Pipeline_1.fit(X_train, y_train)
```



- Make predictions.

```
# get predictions
preds = Pipeline_1.predict(X_valid)
```

- Calculate mean absolute error.

```
print('MAE:', mean_absolute_error(y_valid, preds))
```

MAE: 0.02631578947368421

The mean absolute error is 0.0263157 which is little more than the mean absolute error of the cross-validated model which is 0.026282

- Display accuracy score, confusion matrix and the classification report.

```
print("Accuracy:", accuracy_score(y_valid, preds))
print("Confusion Matrix:\n", confusion_matrix(y_valid, preds))
print("Classification Report:\n", classification_report(y_valid, preds))
```

Accuracy: 0.9736842105263158

Confusion Matrix:

```
[[65  2]
```

```
[ 1 46]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.97	0.98	67
1	0.96	0.98	0.97	47
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

Accuracy is 0.97368

Studying the confusion matrix we can see that

- 65 of actual zero values were predicted correct as 0. Only 2 rows are wrongly predicted as 1.
- 46 of actual one values were predicted correct as 1. Only 1 rows are wrongly predicted as 0.

So, the model has only made 3 errors for 114 records/ rows.

Next the classification report gives the precision, recall and F1 score per class. This is actually the calculated numerical representation of confusion matrix.

Class 0:

Precision = 0.98 out of all predicted 0s, 98% was actually zero.

Recall = 0.97 out of all actual 0s, 97% was correctly predicted.

F1-score = 0.98 mean of precision and recall

Support = 67 number of actual 0 values

Class 1:

Precision = 0.96 out of all predicted 1s, 96% was actually one.

Recall = 0.98 out of all actual 1s, 98% was correctly predicted.

F1-score = 0.97 mean of precision and recall

Support = 47 number of actual 1 values

Overall accuracy is 0.97368

Overall mean absolute error is 0.2631

The model is performing very well on both classes with no bias.

The Accuracy is 0.97368 which is little less than the Accuracy of the cross-validated model which is 0.97371.

- Define a model with parameter `n_estimators` which refers to number of models that included in the ensemble or the times of iterations and parameter `learning_rate`.
- Training the model with parameter `early_stopping_rounds` which offers a way to automatically find the ideal value for `n_estimators`.

```
# Define the model
my_model_2 = XGBClassifier(n_estimators=1000, learning_rate=0.05)

# Fit the model
my_model_2.fit(X_train, y_train,
               early_stopping_rounds=5,
               eval_set=[(X_valid, y_valid)],
               verbose=False)

# Get predictions
predictions_2 = my_model_2.predict(X_valid)

# Calculate MAE
mae_2 = mean_absolute_error(predictions_2, y_valid)

# Uncomment to print MAE
print("Mean Absolute Error:" , mae_2)
print("Accuracy:", accuracy_score(y_valid, preds))
print("Confusion Matrix:\n", confusion_matrix(y_valid, preds))
print("Classification Report:\n", classification_report(y_valid, preds))
```

```

Mean Absolute Error: 0.03508771929824561
Accuracy: 0.9736842105263158
Confusion Matrix:
[[65  2]
 [ 1 46]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.98	0.97	0.98	67
1	0.96	0.98	0.97	47
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

7. Model Evaluations

7.1 Logistic regression

- Calculating and printing accuracy score. Input = y_test, y_pred
- Calculating and printing confusion matrix. Input = y_test, y_pred
- Calculating and printing classification report. Input = y_test, y_pred

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9736842105263158
Confusion Matrix:
[[70  1]
 [ 2 41]]
Classification Report:
              precision    recall  f1-score   support

     0       0.97       0.99       0.98         71
     1       0.98       0.95       0.96         43

 accuracy          0.97          0.97          0.97        114
 macro avg         0.97          0.97          0.97        114
 weighted avg      0.97          0.97          0.97        114
```

Accuracy is 0.97368421.

Studying the confusion matrix, we can see that

- 70 of actual zero values were predicted correct as 0. Only one row has wrongly predicted as 1.
- 41 of actual one values were predicted correct as 1. Only two rows has wrongly predicted as 0.

So, the model has only made 3 errors for 114 records/ rows.

Next the classification report gives the precision, recall and F1 score per class. This is actually the calculated numerical representation of confusion matrix.

Class 0:

Precision = 0.97 out of all predicted 0s, 97% was actually zero.

Recall = 0.99 out of all actual 0s, 99% was correctly predicted.

F1-score = 0.98 mean of precision and recall

Support = 71 number of actual 0 values

Class 1:

Precision = 0.98 out of all predicted 1s, 98% was actually one.

Recall = 0.95 out of all actual 1s, 95% was correctly predicted.

F1-score = 0.96 mean of precision and recall

Support = 43 number of actual 1 values

Overall accuracy is 0.976

The model is performing very well on both classes with no bias. To make sure that these high results are not an effect of overfitting we did a accuracy test on training data.

```
# Predictions on training data
y_train_pred = model.predict(X_train)

# Training accuracy
train_accuracy = accuracy_score(y_train, y_train_pred)
print("Training Accuracy:", train_accuracy)

# Full training classification report
print("\nTraining Classification Report:")
print(classification_report(y_train, y_train_pred))

# Confusion matrix for training set
print("\nTraining Confusion Matrix:")
print(confusion_matrix(y_train, y_train_pred))
```

Training Accuracy: 0.9868131868131869

Training Classification Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	286
1	0.99	0.98	0.98	169
accuracy			0.99	455
macro avg	0.99	0.98	0.99	455
weighted avg	0.99	0.99	0.99	455

Training Confusion Matrix:

```
[[284  2]
 [ 4 165]]
```

Training accuracy = 0.98

Testing accuracy = 0.97

Conclusions:

Since both training and test accuracies are consistently high, it is safe to assume that this model is not overfitting. **The model is performing very well on both classes.**

7.2 Random Forest

- Calculating the mean absolute error between the prediction values and the actual values.

```
print('MAE:', mean_absolute_error(y_valid, preds))
```

```
MAE: 0.03508771929824561
```

The Random Forest has a mean absolute error of 0.03508. Which means the model performs quite well.

- Display accuracy score, confusion matrix and the classification report.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
✓ 0.0s
```

```
print("Accuracy:", accuracy_score(y_valid, preds))  
print("Confusion Matrix:\n", confusion_matrix(y_valid, preds))  
print("Classification Report:\n", classification_report(y_valid, preds))
```

```
✓ 0.0s
```

```
Accuracy: 0.9649122807017544
```

```
Confusion Matrix:
```

```
[[64  3]
```

```
 [ 1 46]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.98	0.96	0.97	67
1	0.94	0.98	0.96	47
accuracy			0.96	114
macro avg	0.96	0.97	0.96	114
weighted avg	0.97	0.96	0.97	114

Accuracy is 0.96491,

Studying the confusion matrix we can see that

- 64 of actual zero values were predicted correct as 0. Only 3 rows are wrongly predicted as 1.
- 46 of actual one values were predicted correct as 1. Only one row has wrongly predicted as 0.

So, the model has only made 4 errors for 114 records/ rows.

Next the classification report gives the precision, recall and F1 score per class. This is actually the calculated numerical representation of confusion matrix.

Class 0:

Precision = 0.98 out of all predicted 0s, 98% was actually zero.

Recall = 0.96 out of all actual 0s, 96% was correctly predicted.

F1-score = 0.97 mean of precision and recall

Support = 67 number of actual 0 values

Class 1:

Precision = 0.94 out of all predicted 1s, 94% was actually one.

Recall = 0.98 out of all actual 1s, 98% was correctly predicted.

F1-score = 0.96 mean of precision and recall

Support = 47 number of actual 1 values

Overall accuracy is 0.9649

The model is performing very well on both classes with no bias.

7.3 Random Forest with Cross validation

- Executing the function with best number of trees.

```
# Execute the get_score_final() with the best n_estimator value
get_score_final(140)
```

✓ 13.3s

Cross-validated Accuracy: 0.9737170387576891
Cross-validated MAE: 0.026282961242310835

Confusion Matrix:
[[352 5]
 [10 202]]

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	357
1	0.98	0.95	0.96	212
accuracy			0.97	569
macro avg	0.97	0.97	0.97	569
weighted avg	0.97	0.97	0.97	569

Accuracy is 0.97371

Studying the confusion matrix we can see that

- 352 of actual zero values were predicted correct as 0. Only 5 rows are wrongly predicted as 1.
- 202 of actual one values were predicted correct as 1. Only 10 rows are wrongly predicted as 0.

So, the model has only made 15 errors for 569 records/ rows.

As the above result a Conclusion can be made that the model is performing well.

Next the classification report gives the precision, recall and F1 score per class. This is actually the calculated numerical representation of confusion matrix.

Class 0:

Precision = 0.97 out of all predicted 0s, 97% was actually zero.

Recall = 0.99 out of all actual 0s, 99% was correctly predicted.

F1-score = 0.98 mean of precision and recall

Support = 357 number of actual 0 values

Class 1:

Precision = 0.98 out of all predicted 1s, 98% was actually one.

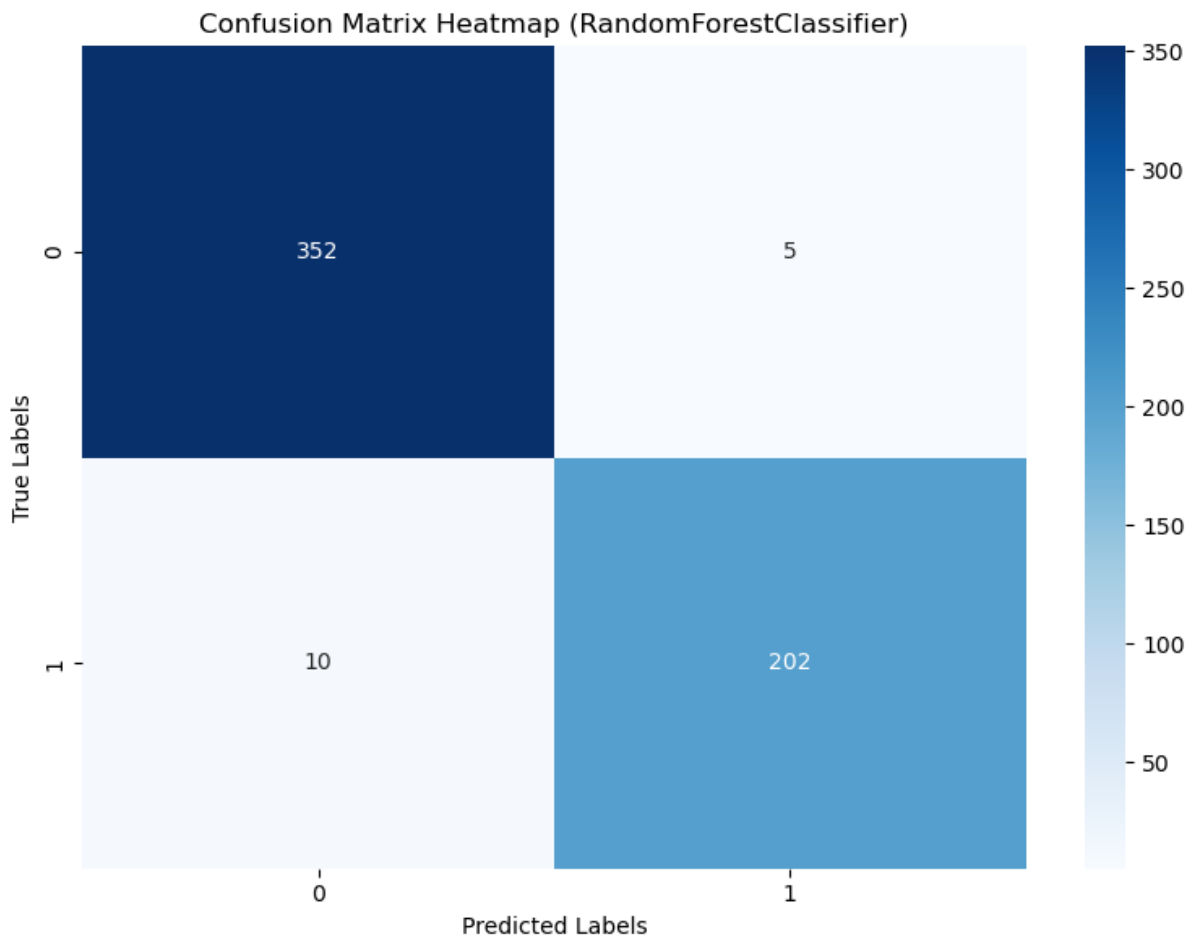
Recall = 0.95 out of all actual 1s, 95% was correctly predicted.

F1-score = 0.96 mean of precision and recall

Support = 212 number of actual 1 values

Overall accuracy is 0.9737

The model is performing very well on both classes with no bias.



The mean absolute error is 0.026282961 and the final accuracy is 0.9737 which is higher than the normal random forest classifier model.

7.4 Model with gradient boosting

- Display mean absolute error, accuracy score, confusion matrix and the classification report.

```
Mean Absolute Error: 0.03508771929824561
Accuracy: 0.9736842105263158
Confusion Matrix:
[[65  2]
 [ 1 46]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.98	0.97	0.98	67
1	0.96	0.98	0.97	47
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

Accuracy is 0.97368

Studying the confusion matrix we can see that

- 65 of actual zero values were predicted correct as 0. Only 2 rows are wrongly predicted as 1.
- 46 of actual one values were predicted correct as 1. Only 1 rows are wrongly predicted as 0.

So, the model has only made 3 errors for 114 records/ rows.

Next the classification report gives the precision, recall and F1 score per class. This is actually the calculated numerical representation of confusion matrix.

Class 0:

Precision = 0.98 out of all predicted 0s, 98% was actually zero.

Recall = 0.97 out of all actual 0s, 97% was correctly predicted.

F1-score = 0.98 mean of precision and recall

Support = 67 number of actual 0 values

Class 1:

Precision = 0.96 out of all predicted 1s, 96% was actually one.

Recall = 0.98 out of all actual 1s, 98% was correctly predicted.

F1-score = 0.97 mean of precision and recall

Support = 47 number of actual 1 values

Overall accuracy is 0.97368

Overall mean absolute error is 0.035087

The model is performing very well on both classes with no bias.

Model	Accuracy	MAE
Logistic Regression (testing)	0.9736	0.0263
Random Forest (default)	0.9649	0.0350
Random Forest (cross-validated)	0.9737	0.0262
Gradient Boosting (dropped columns)	0.9736	0.0263

After considering results of all classification models on the breast cancer diagnosis dataset, the Cross validated Random Forest Classifier was selected as the most suitable model regarding its lower mean absolute error which is 0.0262 and higher accuracy 0.9737.

This model achieved closely the best performance while maintain robustness against overfitting and data leakage. Also, it can consider reliable and interpretable.

Random Forest as its nature trains number of decision trees to get an average of their results, make it suitable for deal with more complex feature interacted data sets. Meanwhile Cross validation ensured that the models performance consistency with different subsets of the data, which will reduce the risk of overfitting and more generalize for unseen data, which makes the model is accurate for real world scenarios.

With this Random Forest and Cross Validation combination the most accurate and trustworthy diagnosis predictions can be given to the world in the sake of humanity.

8. Deployment

The process of making a machine learning model accessible for users and applications is called as deployment. It is the final step of setting up a machine learning model so other users and application can get predication from this model. To deploy a model, first step is packaging the trained model after successfully packaging model create API endpoints and hosting it on cloud service platform helps users to interact with model. This process involves a series of steps to ensure the model is robust, scalable and reliable.

1. Model Serializing and Packaging

- After training model first step is to save the model in a format that can be easily to loaded later. This process called as serializing. Standard way to do this in python is use **pickle** or **joblib** library.
- The trained model object is saved to a file with **.pkl** extension. This file contains all the learning parameters and the model's structure.
- To make sure model run in any environment it needs necessary dependencies and their versions. Creating requirement.txt file ensure that model will reproducing that exact environment where the model was trained.

2. API Development

- Allow other users and applications to send data and receive data need API. This API act like middleman, handling incoming requests, processing data, making a prediction with loaded model and send the results back. This needs a lightweight web framework like FastAPI or Flask.
- To setup the framework, a simple python script can be used to define the endpoints. After setting up the API framework, the saved model file can be loaded into the python script. This step should be done only once when the server starts.

3. Web Application

- This machine learning model focuses more on the lab research side, so to make it easy for lab assistants to use, a simple and user-friendly web application will be implemented. This application would allow users to input the data through a text box also to show the predicted value there will be simple user interface that user can simply go through it and get that predicted value.

4. Containerization

- Ensure model runs consistently and regardless of the server environment, containerizing the model using Docker is very use full not only that this helps model to scalable and robust deployment. In Docker it will container packages application, dependencies and its configurations into single isolated unit.
- When containerizing the application first need a Docker image. To create a docker image base of our application, it needs a **Dockerfile**. This fil contains instructions for build the docker container.

5. Hosting and Scaling

- Finally make it accessible to other users over the internet we need to host our containerized application on a server. Cloud platforms offer various services for host these types of applications.
- Platforms like AWS, Google Cloud Platform or Microsoft Azure provides services like AWS EC2, GCP Cloud Run, Azure App Service. To deploy our application, we choose AWS EC2 services because AWS is a leading cloud computing platform. It is more reliable, scalable, flexible, cost effective and it has more security features.

9. Conclusion

Machine learning techniques exist for predicting breast cancer diagnosis through medical diagnosis. The system identifies breast cancer tumors as either benign or malignant through classification. The disease of breast cancer stands as one of the leading causes of worldwide mortality rates. The detection of cancer at an early stage through medical screening methods leads to life-saving outcomes. The existing diagnostic methods show strong performance yet they require human operators and produce high costs and need invasive procedures. The development of automated machine learning tools enables healthcare professionals to obtain fast and scalable accurate clinical decision support.

In this research, we used a Kaggle breast cancer dataset with 569 records, publicly available and containing features extracted from digitized images of fine-needle aspirates. The data was cleaned, irrelevant features were dropped, the target was encoded, and the values were scaled. By using various statistical tests and a lot of visualizations, we were able to see which features were useful in making a diagnosis. As a result, the final models were trained on useful data to ensure that not much noise or redundancy is there.

We used Logistic Regression for baseline and Random Forest for main predictive model. These models were built initially. While Logistic Regression was useful for testing, the Random Forest machine learning algorithm outperformed due to its ensemble nature, which captures complex interactions between features without suffering from overfitting. Through metrics such as mean absolute error, accuracy score, confusion matrix, and classification reports, Random Forest showed to be a good fit for the classification. The accuracy of the Random Forest model was found to be 96.49% at first but then 97.37% using CV tuning.

By changing the number of decision trees and utilizing gradient boosting, we learned more through our experiments. In some cases, gradient boosting performed slightly better (98.24%). However, these results may be overfitting. The Random Forest model was found to be the best performing model considering both accuracy and generalizability after cross-validation.

Finally, the study also examined deployment strategies. The trained model can be deployed to real world systems by serializing the model, creating API endpoints using Flask, FastAPI or similar frameworks and containerizing using Docker. The medical professionals can host the application on the cloud, like AWS EC2, for better scaling.

Overall, we got to see all the stages of generating a machine learning solution for breast cancer prediction. Through this project, we got familiarized with and executed the process of ML solution generation. The model was selected, preprocessed, built, evaluated, tuned, and deployed by us over the course of the year. Data-driven approaches can help the health sectors to an extent, if the results suggest so, for sure. In addition, it will promote traditional medical methods. As it provides faster, and more accurate and more consistent diagnostics value and support. No model can replace the expert himself, though machine learning can be a powerful tool that reduces diagnosis time and enhances decision-making by health care personnel. If you test it on big samples, it can help in health care. Testing it on bigger and varied groups will help it make its contribution.

References

- AlexisBCook. (n.d.). *XGBoost*. Retrieved from [www.kaggle.com: https://www.kaggle.com/code/alexisbcook/xgboost](https://www.kaggle.com/code/alexisbcook/xgboost)
- cook, A. (n.d.). */code/alexisbcook/data-leakage*. Retrieved from [Kaggle.com: https://www.kaggle.com/code/alexisbcook/data-leakage](https://www.kaggle.com/code/alexisbcook/data-leakage)
- G, M., Alkabban, & Ferguson. (2024, 2 25). Retrieved from [www.ncbi.nlm.nih.gov: https://www.ncbi.nlm.nih.gov/books/NBK482286](https://www.ncbi.nlm.nih.gov/books/NBK482286)
- Momeni, M. (n.d.). */datasets/imtkaggleteam/breast-cancer*. Retrieved from [Kaggle.com: https://www.kaggle.com/datasets/imtkaggleteam/breast-cancer](https://www.kaggle.com/datasets/imtkaggleteam/breast-cancer)
- Organization, W. H. (n.d.). Retrieved from <https://www.who.int>: https://www.who.int/news-room/fact-sheets/detail/breast-cancer?utm_source=chatgpt.com